

2

Módulo 02 - Captura e processamento de vídeo

Visão de Computador e Stable Diffusion

Resumo do Módulo

- ◆ Captura e exibição de vídeo (VideoCapture, imshow).
- ◆ Análise de propriedades do vídeo (largura, altura, fps).
- ◆ Processamento de frames (transformações, conversões de cor).
- ◆ Detecção simples de objetos (baseada em cor, template matching, Haar).
- ◆ Desenho de caixas e rótulos em objetos detetados.
- ◆ Gravação do resultado (VideoWriter) em pipeline integrado com OpenCV.
- ◆ Lab

`cv2.VideoCapture()`

- ♦ O objeto de *VideoCapture* permite-nos manipular frames de uma câmara.

`cap = cv2.VideoCapture(0)`

- ♦ O argumento da função `cv2.VideoCapture()` é:

1. Ou o *nome do ficheiro* de vídeo
2. Ou o *índice do hardware* da câmara: **0**,1,2...

- ♦ Depois de utilizada, a captura tem de ser libertada:

`cap.release()`

Definir resolução

```
import cv2  
cap = cv2.VideoCapture(0)  
  
# definir resolução 720p  
cap.set(3, 1280)  
cap.set(4, 720)  
  
#OU definir resolução 480p  
cap.set(3, 640)  
cap.set(4, 480)
```

Usar o feed da webcam

```
cap = cv2.VideoCapture(0)
while(True):
    # Captura frame-by-frame (quadro-a-quadro)
    ret, frame = cap.read()

    # As nossas operações na frame são colocadas aqui
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Exibir o quadro resultante
    cv2.imshow('frame',gray)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

# Quando tudo estiver pronto, libertar a captura
cap.release()
```

Importar um vídeo

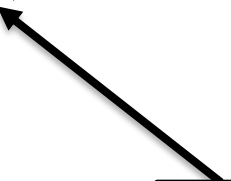
```
cap = cv2.VideoCapture('vtest.avi')
while(cap.isOpened()): #retorna true quando há outro frame

    #para processar
    ret, frame = cap.read()

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    cv2.imshow('frame',gray)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
```



1 ms (1000 frames /s)

1000 frames por segundo



cv2.VideoWriter()

- ♦ O objeto de **VideoWriter** permite guardar um vídeo num arquivo de saída.

```
out = cv2.VideoWriter('output.avi', fourcc, 20.0,  
    (640, 480))
```

FourCC é um código de 4 bytes usado para especificar o codec de vídeo

A lista de códigos disponíveis pode ser encontrada em <https://fourcc.org/>

- ♦ (dependente da plataforma)

- ♦ *Depois de utilizada*, a câmara *tem de ser libertada*:

```
out.release()
```


Ex 01: Capturar Vídeo da Câmara

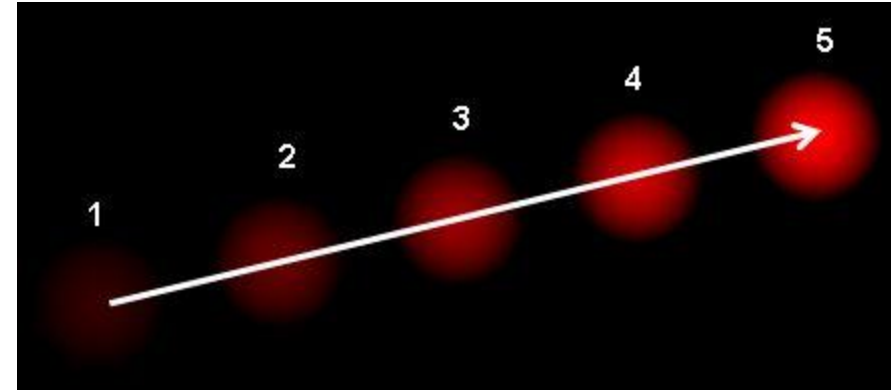
Ex 02: Reproduzir Vídeo de Ficheiro

Ex 03: Gravar Vídeo da Câmara

- ♦ No processamento de imagens, é obrigatório que o código não esteja apenas fornecendo a solução correta, mas também da maneira mais rápida.
- ♦ Funções úteis para medir o desempenho do código.
 - `cv2.getTickCount`
 - `cv2.getTickFrequency`

Fluxo ótico

- ◆ Padrão de *movimento aparente de objetos* de imagem entre dois quadros consecutivos causado pelo movimento de objeto ou câmera.



- ◆ O fluxo ótico tem muitas aplicações em áreas como:
 - Estrutura a partir do movimento
 - Compressão de vídeo
 - Estabilização de vídeo
- ◆ `cv2.calcOpticalFlowPyrLK()`
 - ◆ (Lucas-Kanade method)

Subtração de fundo



Calibração da câmara

- ◆ Câmaras baratas introduz muita distorção nas imagens. 2 distorções principais são a *distorção radial* e a *distorção tangencial*.

É necessário remapear a **leitura da Matriz**

- ◆ `cv2.findChessboardCorners` – Dummy Test
- ◆ `cv2.calibrateCamera()`
- ◆ `cv2.getOptimalNewCameraMatrix()`

Modelo da câmara Pinhole: Camera Matrix

- ♦ **Camera matrix:** uma matriz 3x4 que se transforma de pontos de cena 3D (x, y, z) para pontos de tela 2D $(x'/w', y'/w')$:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & c_{13} & c_{14} \\ c_{21} & c_{22} & c_{23} & c_{24} \\ c_{31} & c_{32} & c_{33} & c_{34} \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

- ♦ Quantos parâmetros livres na matriz da câmara?

Modelo de câmara Pinhole

- ◆ Pode reescrever a matriz da câmara em termos de parâmetros intrínsecos e extrínsecos.

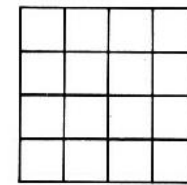
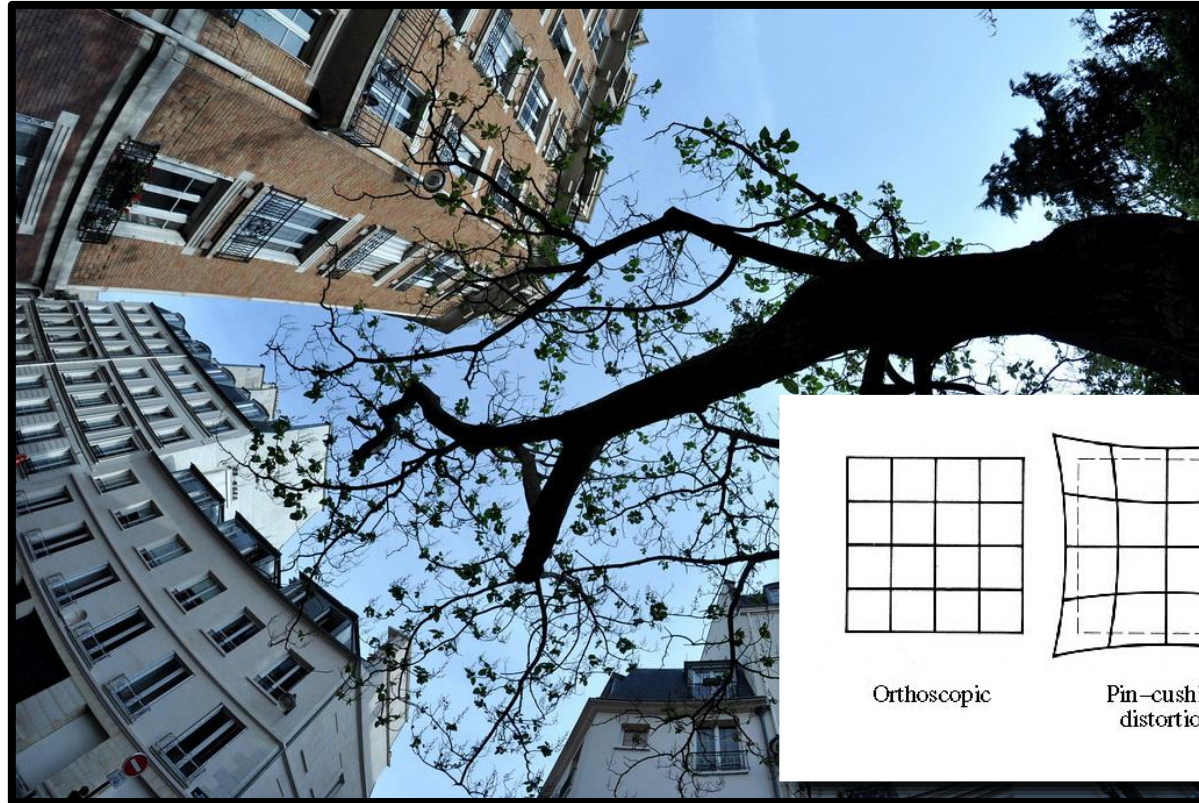
$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \mathbf{K} [\mathbf{R} \quad \mathbf{t}] \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$$\mathbf{K} = \begin{bmatrix} \alpha_x & \gamma & u_0 \\ 0 & \alpha_x & v_0 \\ 0 & 0 & 1 \end{bmatrix}$$

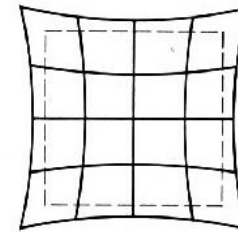
Intrinsic matrix

- ◆ Quantos parâmetros intrínsecos?
- ◆ Quantos parâmetros extrínsecos?
- ◆ Quantos parâmetros no total?

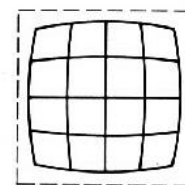
Modelo de câmara real: Distorção radial



Orthoscopic



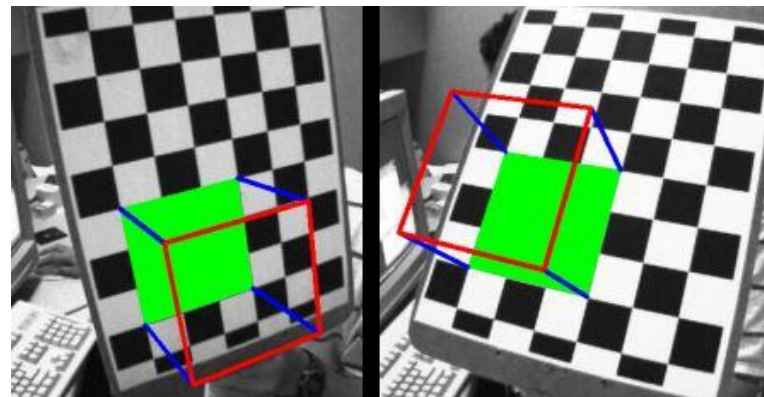
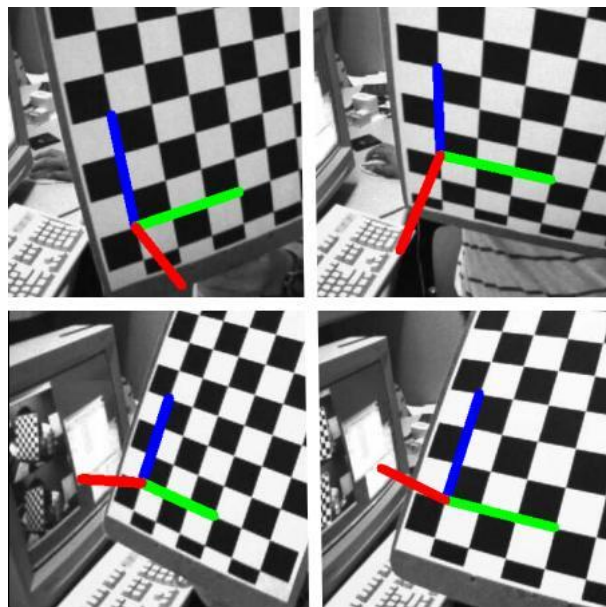
Pin-cushion
distortion



Barrel
distortion

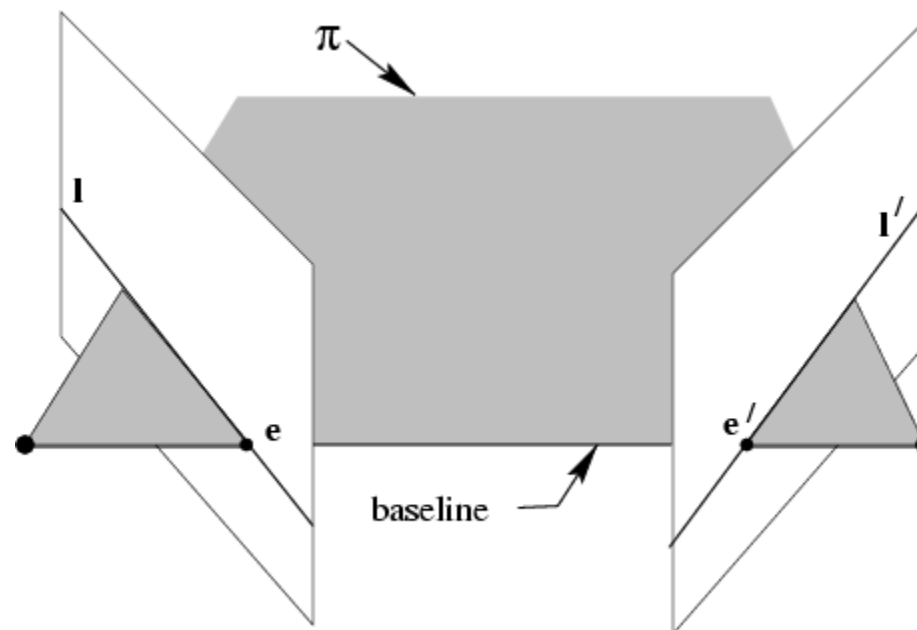
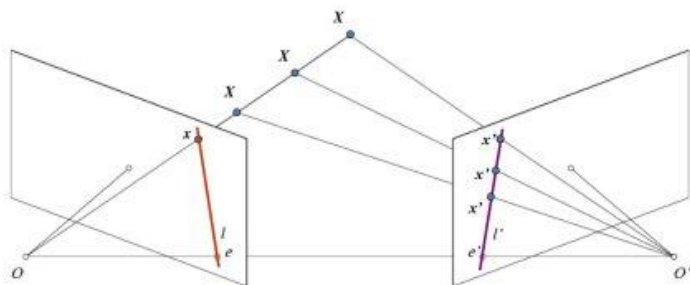
Estimativa de pose

- ◆ Use a matriz da câmara e o módulo `calib3d` para criar alguns efeitos 3D na imagem

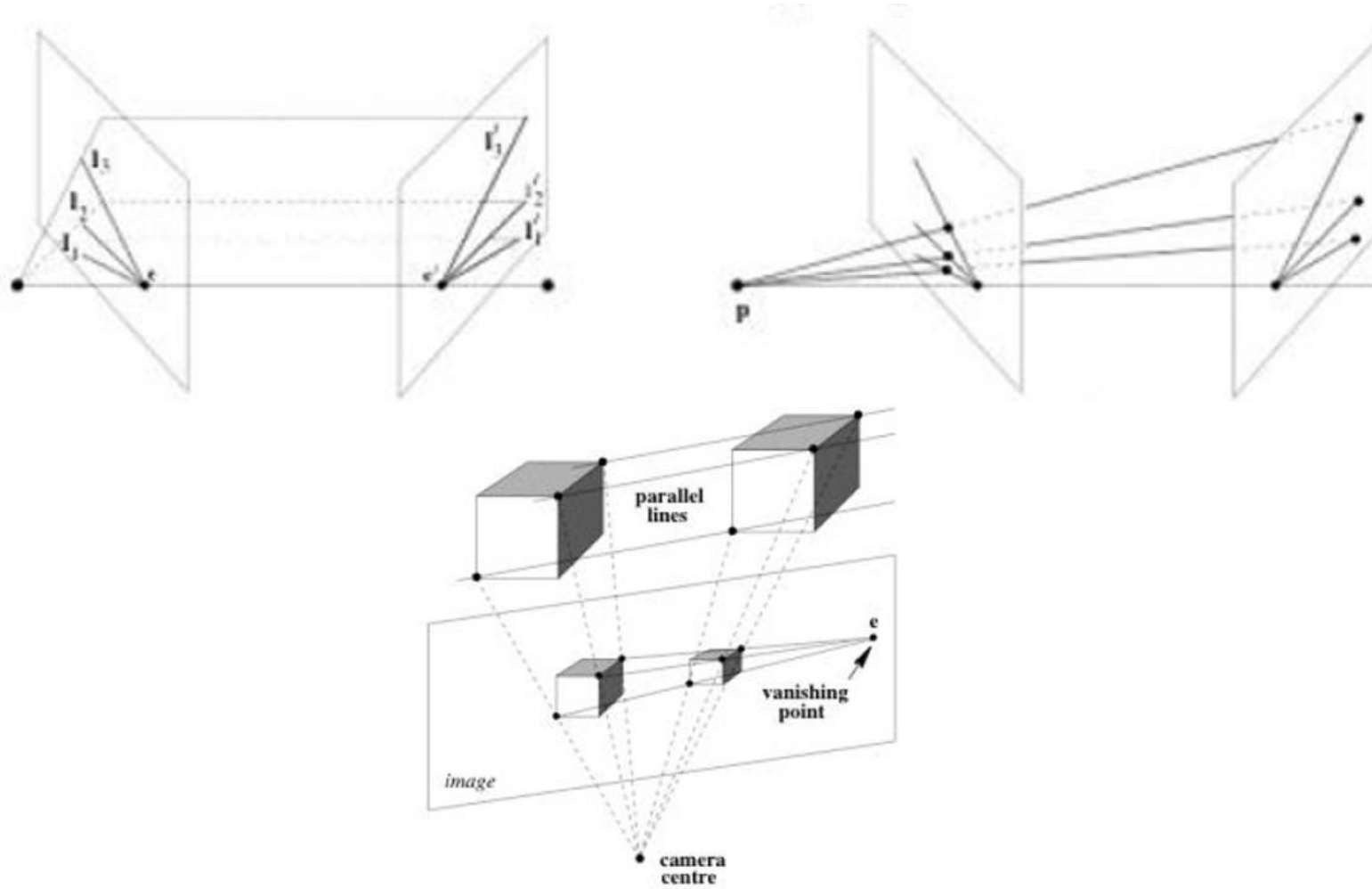


Geometria Epipolar

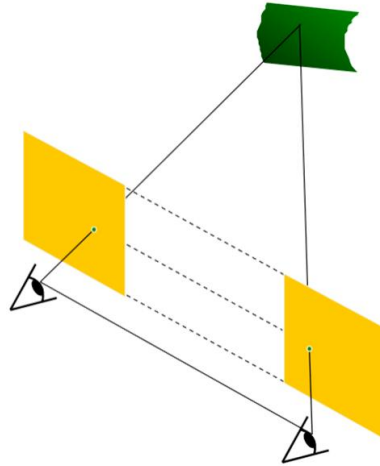
1. **Linha de base**: conecta dois centros de câmara
2. **Epípole**: ponto de intersecção da linha de base com o plano da imagem
3. **Epipole**: imagem em uma visão do centro da câmara da outra vista.



Geometria Epipolar – Mais exemplos

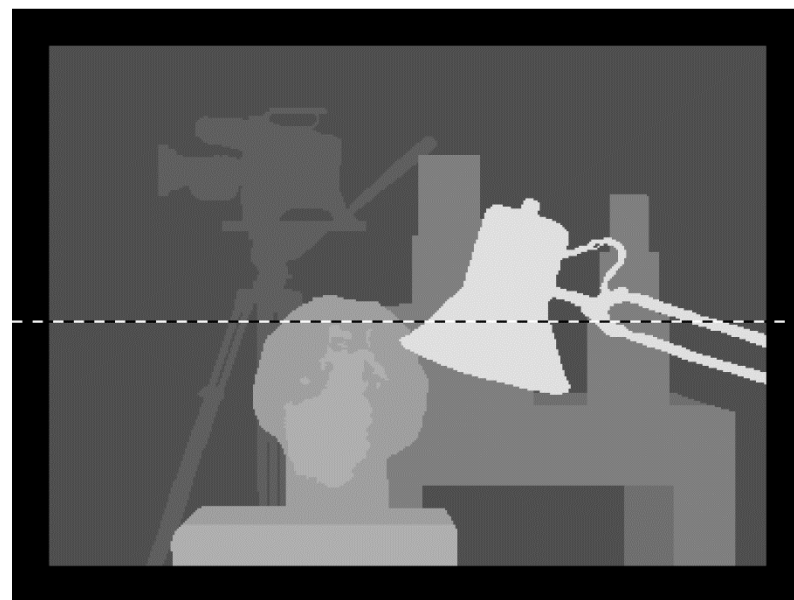


Imagens estéreo

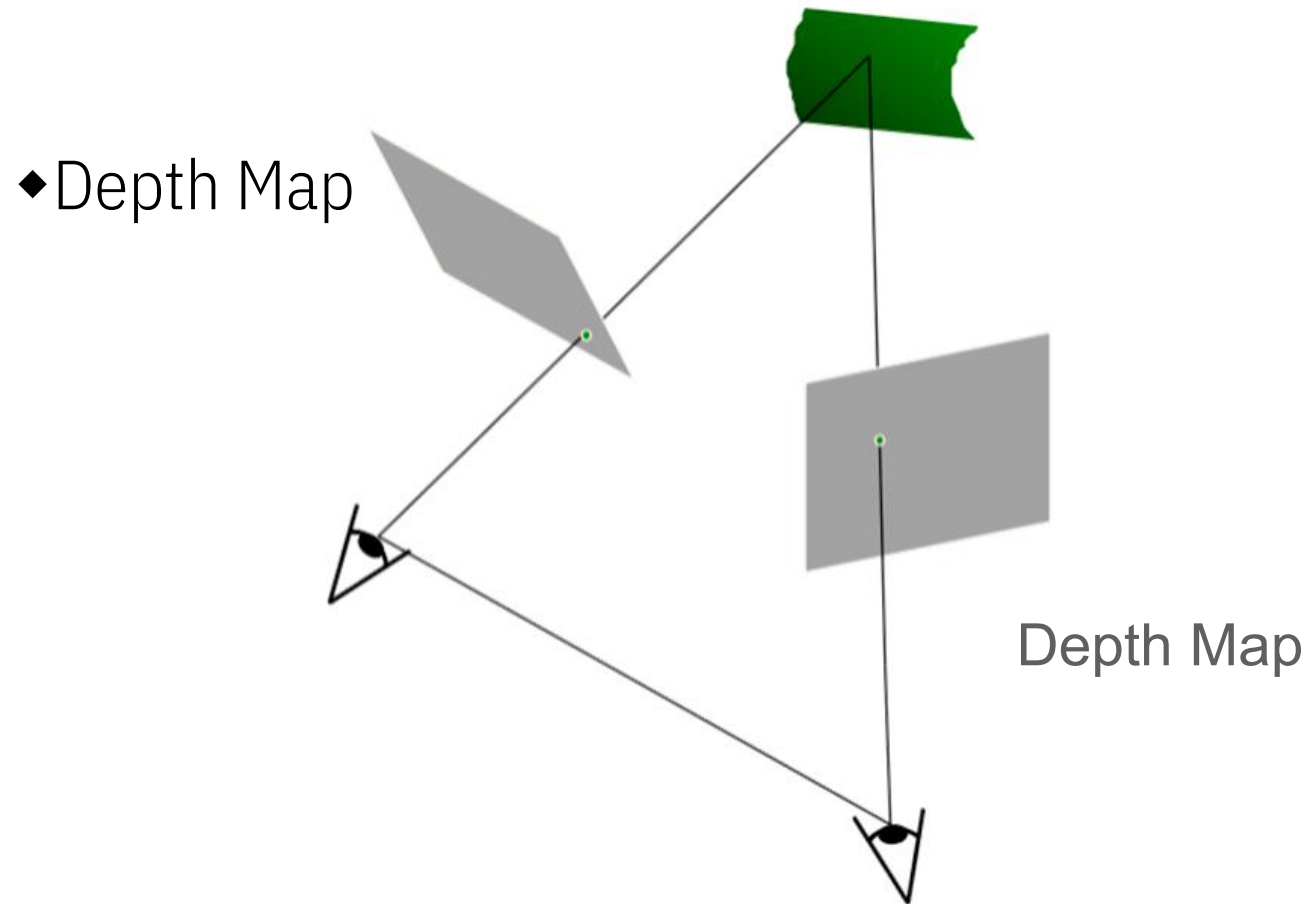


Mapa de profundidade a partir de imagens estéreo

- ♦ A imagem abaixo contém a imagem original (*esquerda*) e seu mapa de disparidade (*direita*). O resultado está contaminado com alto grau de ruído.
- ♦ Ao ajustar os valores de **numDisparities** e **blockSize**, pode obter melhores resultados.



Retificação estéreo



Ex 04: Detecção de Cor em Tempo Real com HSV

Ex 05: Exibir Informações no Vídeo da Webcam

Ex 06: Subtração de Fundo

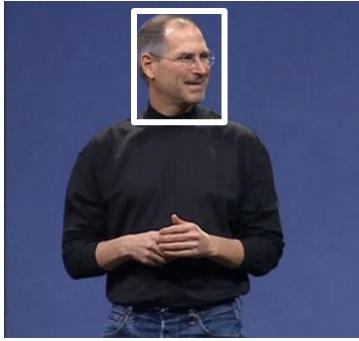
Ex 07: Rastreamento de Pontos usando Optical Flow

Meanshift e Camshift

- ♦ Os algoritmos *Meanshift* e *Camshift* são usados para localizar e rastrear objetos em vídeos.
- `cv2.meanShift()` – usado somente se o *objeto tiver o mesmo tamanho*
- `cv2.CamShift()` – usado mais se o *objeto for redimensionado* na imagem



Face detection



Deteção de rosto $\neq$ Reconhecimento facial
Quem é? $\rightarrow$

$\leftarrow$ Onde está o rosto?



Alfréd Haar

Face Detection - detetar que é um rosto humano

- Haar Classifier
- LBP Classifier

Face Recognition - detetar que é que **aquele** humano

- Fisher
- Eigen
- LBPHF

Object Recognition Classifier

- ♦ Um programa que decide se uma imagem é uma imagem positiva (**face image**) ou imagem negativa(**non-face image**).
- ♦ Um classificador é treinado em centenas de milhares de imagens faciais e não faciais para aprender a classificar uma nova imagem corretamente.

Deteção de rosto

- ♦ O problema da **deteção de rosto** é:
Dada uma imagem, diga se ela contém rostos ou não.
- ♦ A ideia de **deteção de rosto** em visão computacional é deixar o computador aprender a detetar rostos em imagens, assim como um humano pode fazer.

Mas...



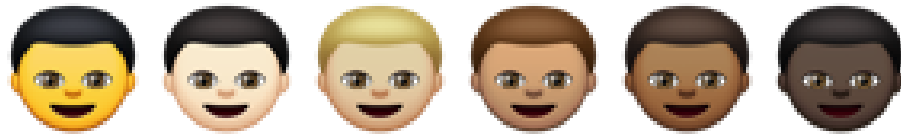
Dificuldades - Mudar a luz

- ◆ Afeta a cor, característica facial



Dificuldades - Tom de Pele

- ♦ Grande variedade de tons de pele.



Dificuldades - Expressões Faciais

- ♦ Afeta a forma do rosto e as suas características



Dificuldades – Escalonamento e Ângulos



Dificuldades - Obstruções

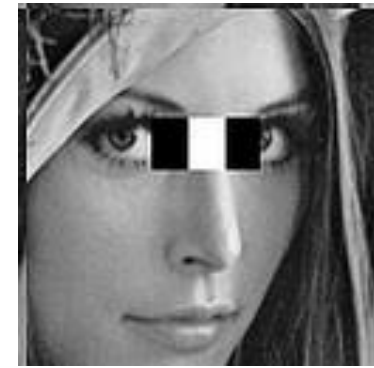
♦Obstrução das características faciais



Haar Cascade

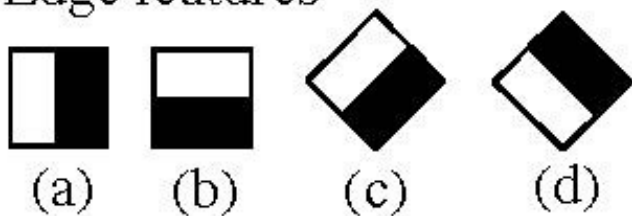
- ◆ Um "cascade" é uma série de "Haar-like features" que são combinados para formar um classificador
- ◆ Haar Cascade = Classifier

Exemplos de como as Fetures Haar correspondem a características faciais

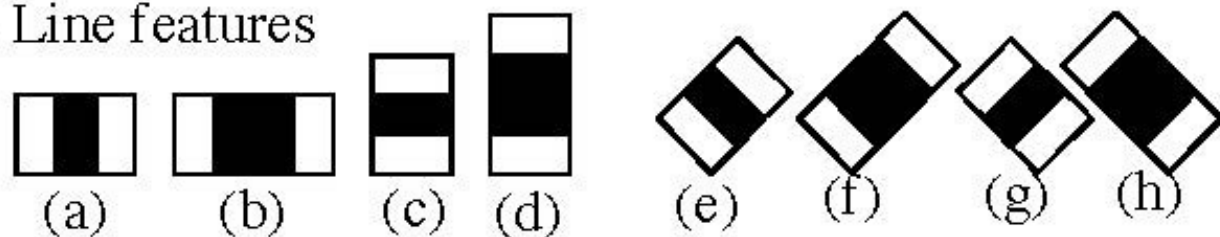


Haar-Like Features

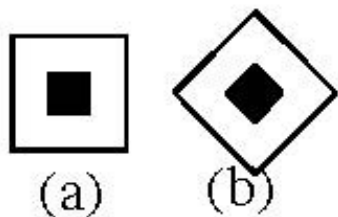
1. Edge features



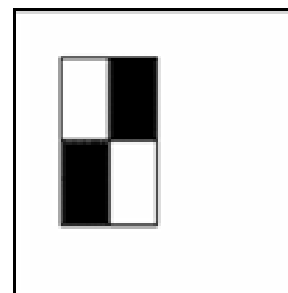
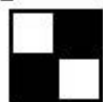
2. Line features



3. Center-surround features



4. Special diagonal line feature used in [3,4,5]



0	0	0	0	0
0	1	-1	0	0
0	-1	1	0	0
0	0	0	0	0

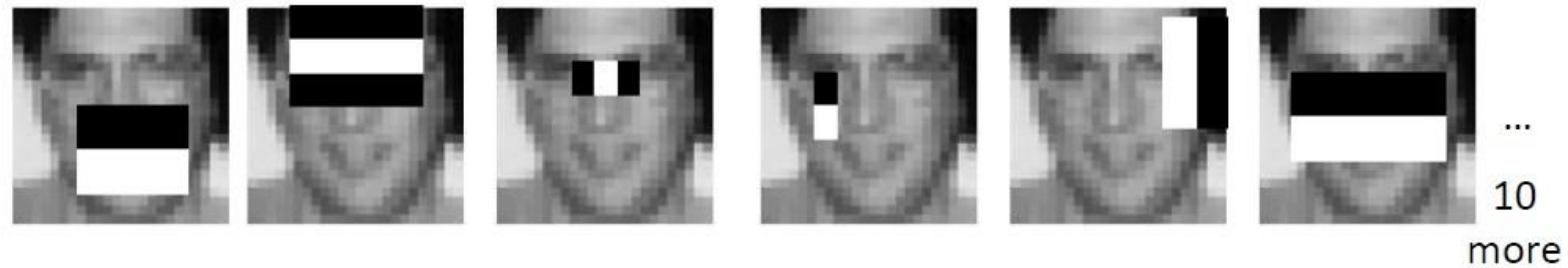
- ♦ O Algoritmo de detecção de objetos Viola–Jones é a primeira estrutura de detecção de objetos a fornecer taxas de detecção de objetos competitivas em tempo real.
 - Primeiro com velocidade!
- ♦ Embora possa ser treinado para *detetar uma variedade de classes de objetos*, foi motivado *principalmente* pelo problema da *detecção de rosto*.
- ♦ O algoritmo tem 4 estágios:
 1. Haar Feature Selection
 2. Creating an Integral Image
 3. Adaboost Training
 4. Final Classifier = Cascading Classifiers

1. Haar Feature Selection

Stage 0

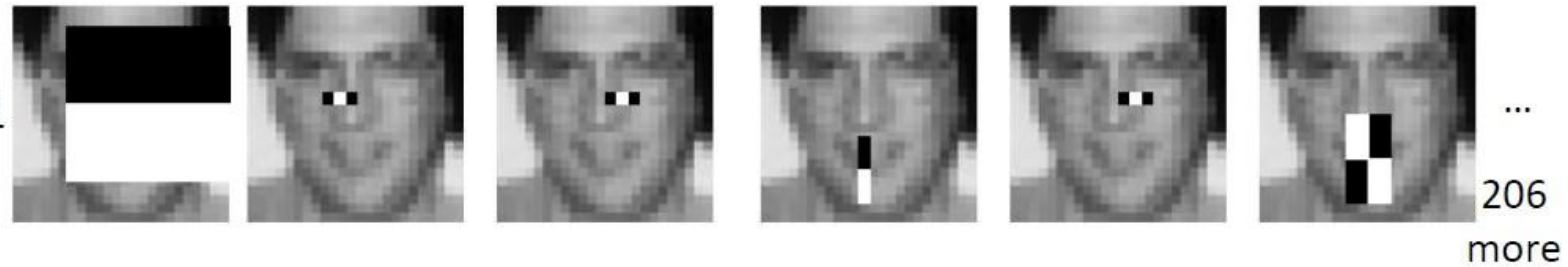


Stage 1



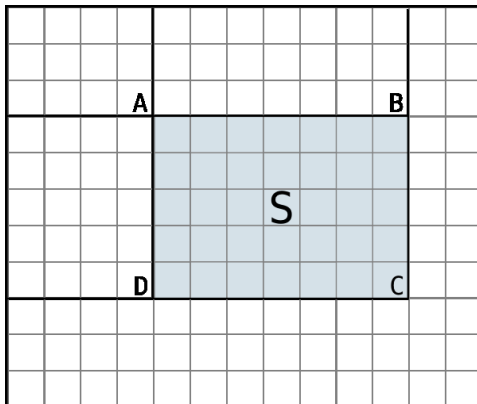
⋮

Stage 21



2. Creating an Integral Image

- ♦ Uma "**summed area table**" é criado em uma única passagem pela imagem.
- ♦ A soma de qualquer região na imagem pode ser calculada por uma única fórmula.



Original				
5	2	3	4	1
1	5	4	2	3
2	2	1	3	4
3	5	6	4	5
4	1	3	2	6

$$5 + 2 + 3 + 1 + 5 + 4 = 20$$

Integral				
5	7	10	14	15
6	13	20	26	30
8	17	25	34	42
11	25	39	52	65
15	30	47	62	81

Original				
5	2	3	4	1
1	5	4	2	3
2	2	1	3	4
3	5	6	4	5
4	1	3	2	6

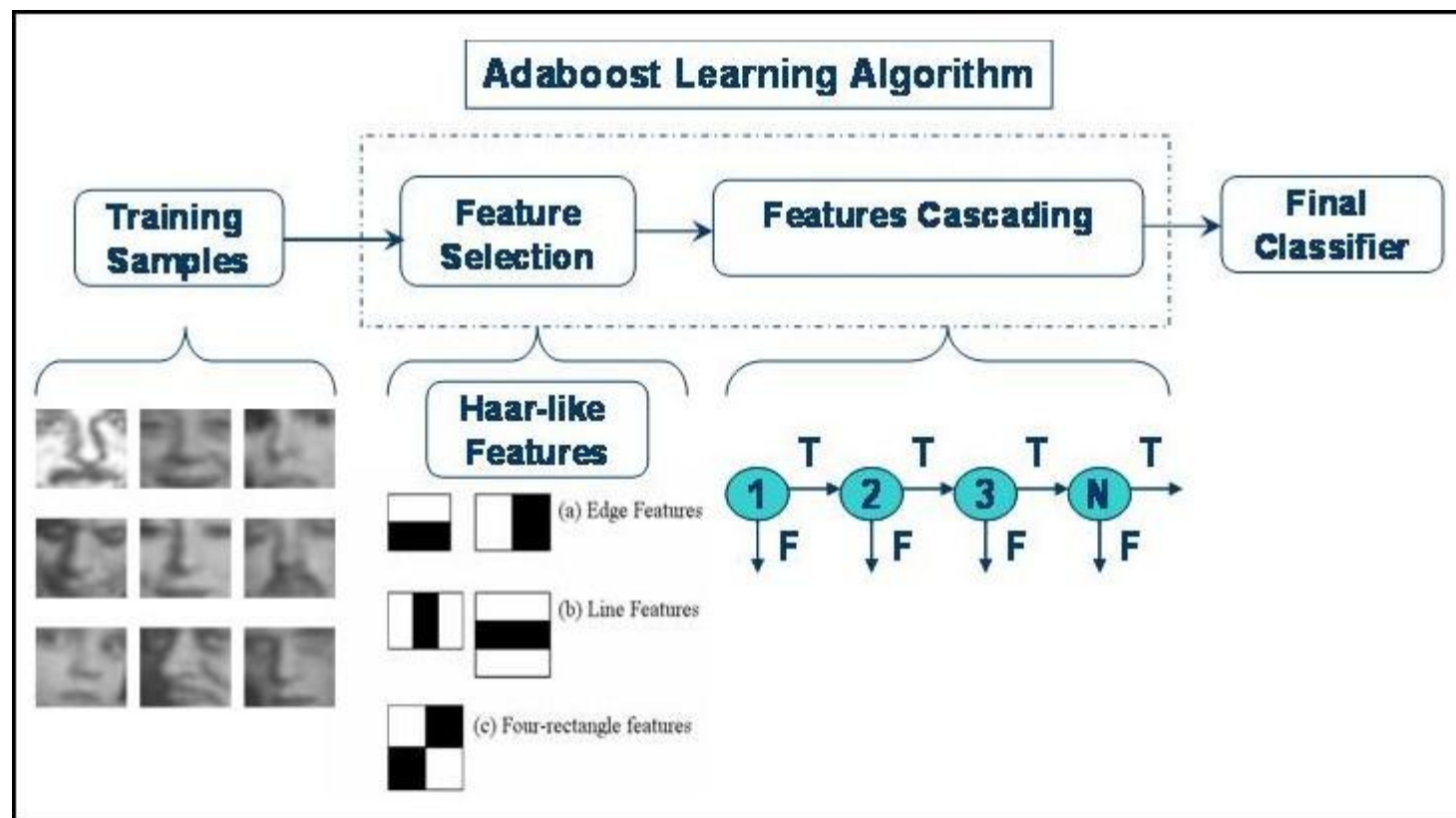
$$5 + 4 + 2 + 2 + 1 + 3 = 17$$

Integral				
5	7	10	14	15
6	13	20	26	30
8	17	25	34	42
11	25	39	52	65
15	30	47	62	81

$$34 - 14 - 8 + 5 = 17$$

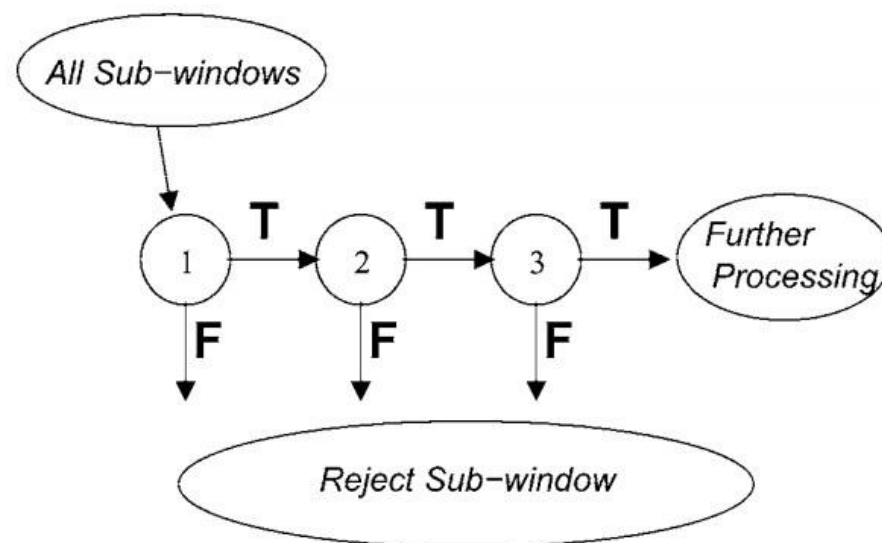
3. AdaBoost (Adaptive Boosting) Training

- ♦ **Adaboost** experimenta vários classificadores fracos ao longo de várias iterações, selecionando o melhor classificador fraco em cada rodada
 - *combinando os melhores classificadores fracos* para criar um classificador forte



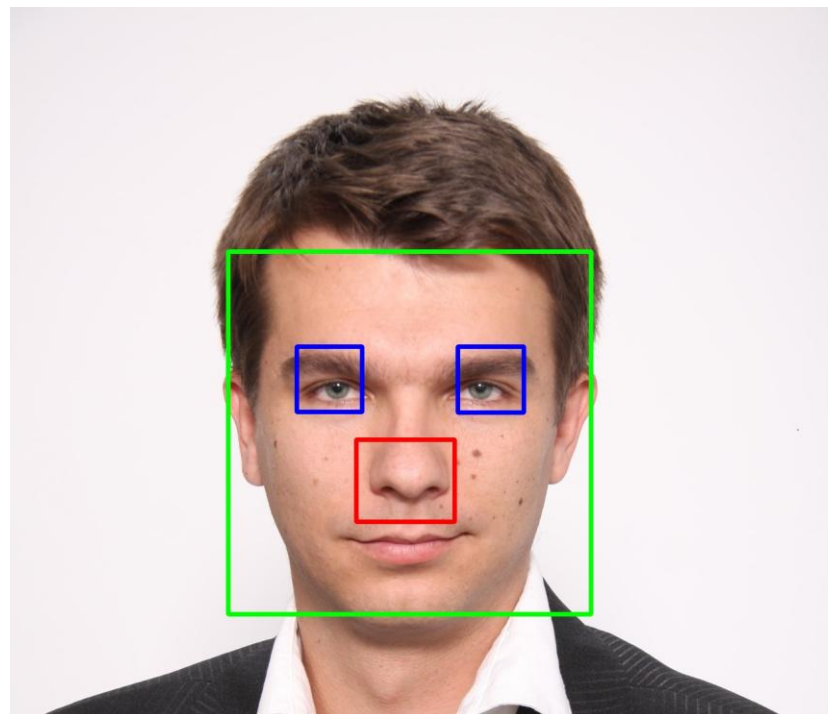
4. Final Classifier = Cascading Classifiers

- ♦ *Haar cascades* consiste em uma *série de classificadores fracos* - aqueles pouco melhores do que 50% corretos
- ♦ Se a área passar por um único classificador, *prossiga*; caso contrário, *não corresponde*



OpenCV já contém muitos classificadores pré-treinados

- Olhos
- Nariz
- Boca
- Rosto
- Sorriso
- Corpo
- Rosto do perfil
- Face frontal
-



Ex 08: Rastreamento de Objeto usando MeanShift

Ex 09: Rastreamento de Objeto usando CamShift

Ex 10: Detecção de Rostos e Olhos com Haar Cascades